

02 Matrices

2.3 Shape of Matrix

From Vectors to Block Matrices: A Tour of Special Shapes



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Understand how column vectors and row vectors are used in data analysis and machine learning

3

Understand why square matrices matter: the determinant, inverse, eigendecomposition, and more all require a square matrix

5

Understand diagonal matrices: every off-diagonal entry is zero, and the matrix can be square or rectangular

7

Understand triangular matrices: upper- and lower-triangular matrices transpose into each other, simplifying equation solving

2

Distinguish tall matrices from wide matrices: more samples than features vs. more features than samples — interchangeable by transpose

4

Master the definition of a symmetric matrix: equal to its own transpose, with entries mirrored across the main diagonal

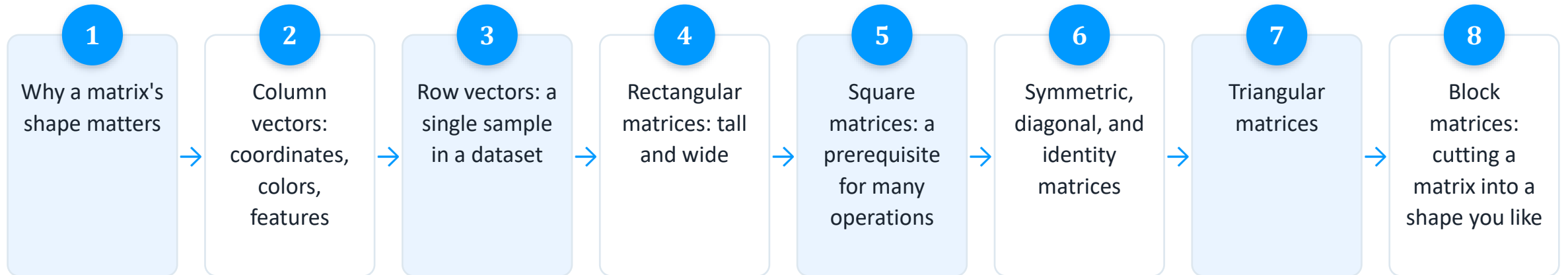
6

Master the identity matrix: a special diagonal matrix with 1s on the main diagonal — the "number 1" of matrix multiplication

8

Master the idea of block matrices: cutting a large matrix into submatrices for more readable, efficient computation

How This Section Unfolds



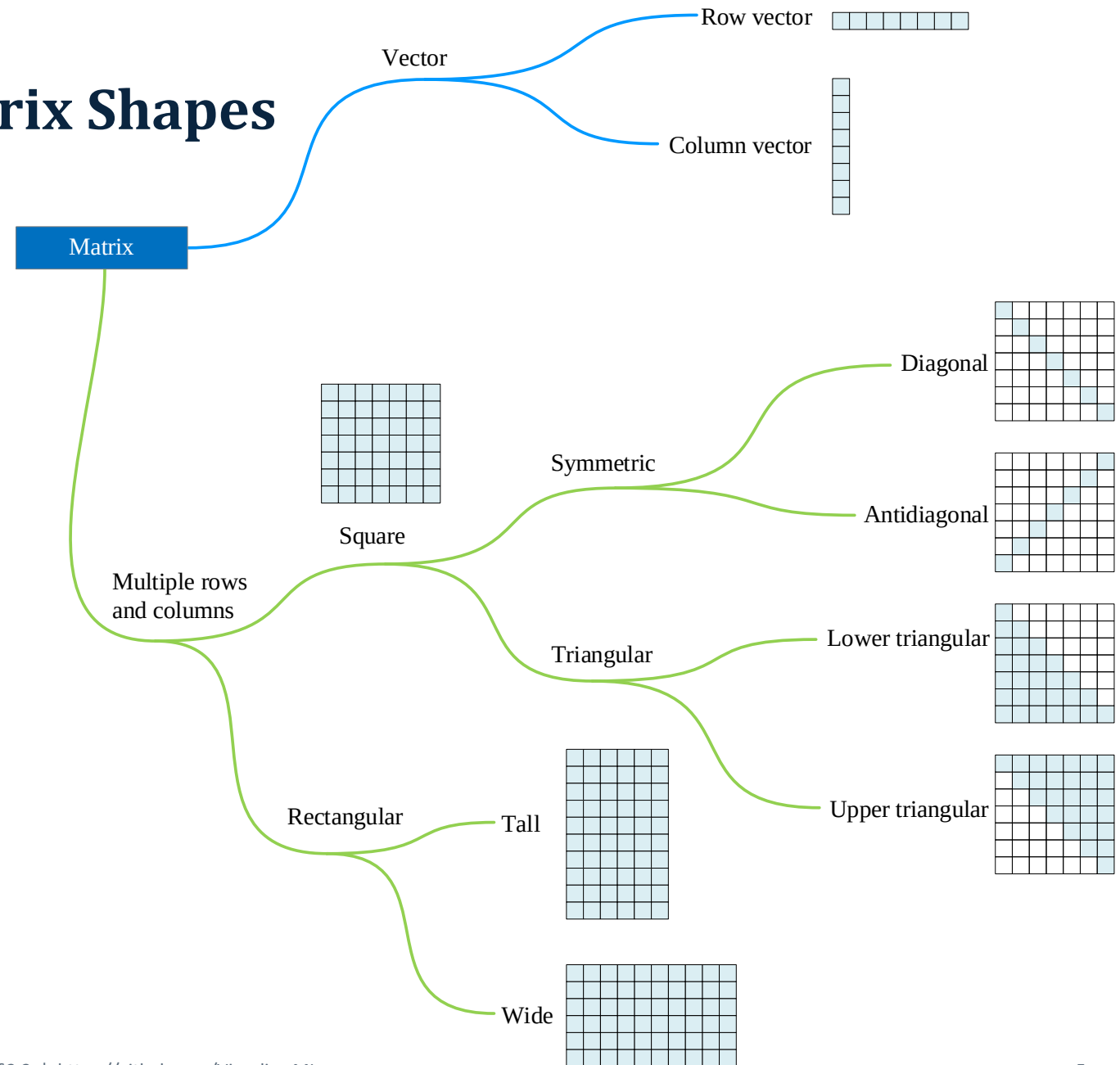
Why Does a Matrix's Shape Matter?

A matrix's shape determines whether it can be added, subtracted, or multiplied — and whether many operations are even defined

- The determinant, LU decomposition, Cholesky decomposition, eigendecomposition, trace, and more all require a square matrix
- Unhappy with a matrix's shape? Block matrices let you cut it into the shape you want

Figure 1 · Several Common Matrix Shapes

- Matrices split into vectors (row/column) and multi-row, multi-column matrices — the latter split into square and rectangular
- Square matrices branch into symmetric, diagonal, triangular; rectangular matrices split into tall and wide



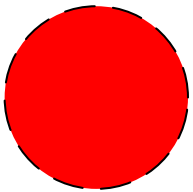
Column Vector: A Matrix with One Column, Many Rows

A row vector transposes into a column vector, and vice versa; this book's vectors default to column vectors

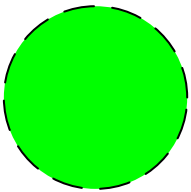
- A point in a plane is written as a 2D column vector $\mathbf{x}=[x_1;x_2]$; a point in 3D space as $\mathbf{x}=[x_1;x_2;x_3]$
- Column vectors are used everywhere in data analysis, linear algebra, and machine learning — the default vector form in this book

Figure 2 · Representing Color with Column Vectors

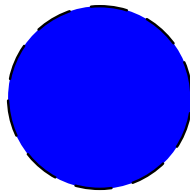
- In RGB space, red, green, and blue are the column vectors $\mathbf{e}_1=[1;0;0]$, $\mathbf{e}_2=[0;1;0]$, $\mathbf{e}_3=[0;0;1]$
- Any color can be written as a 3-component column vector, e.g. yellow $[1;1;0]$, magenta $[1;0;1]$



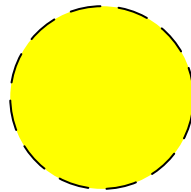
$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$



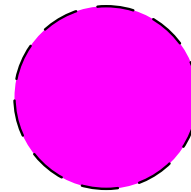
$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$



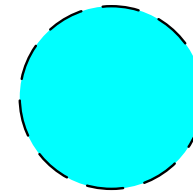
$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$



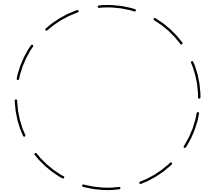
$$\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$



$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$



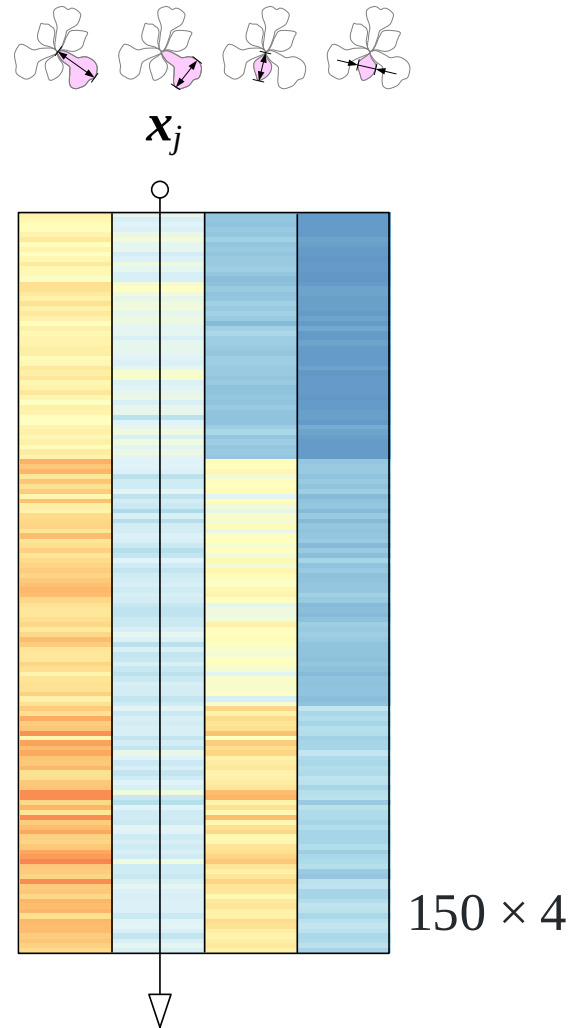
$$\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$



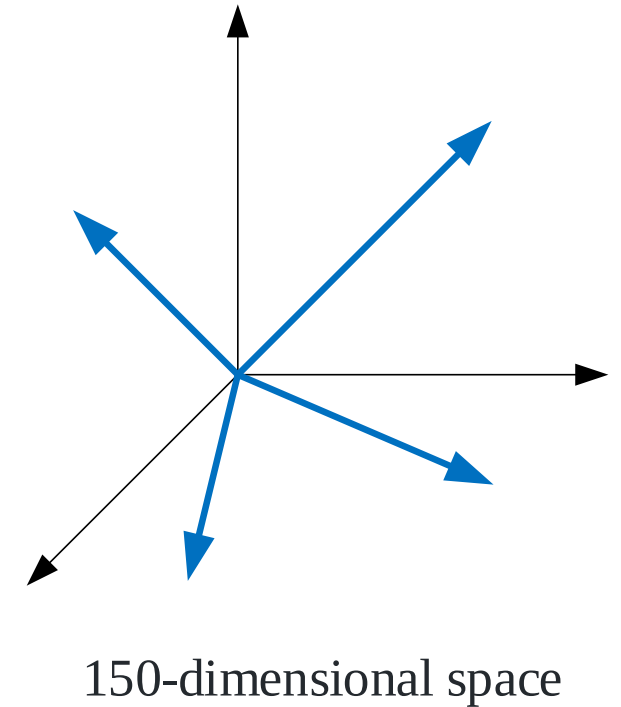
$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Figure 3 · A Column Vector in the Iris Dataset

- In a data matrix, a column vector often stores one feature's data, such as every sample's sepal length
- Column vectors support statistics like mean and variance, or plotting a distribution — the basis of feature engineering



Column vectors



Column Vectors Are What Linear Transformations Act On

A linear transformation preserves vector addition and scalar multiplication — rotation, scaling, reflection are examples

- Unless stated otherwise, every vector in this book defaults to a column vector
- Understanding linear transformations requires matrix multiplication first — the topic of the next section

Row Vector: A Matrix with One Row, Many Columns

A row vector commonly represents a single sample in a dataset, with each column holding one feature

- In the Iris dataset, row 1 is the row vector $\mathbf{x}^{(1)}=[5.1, 3.5, 1.4, 0.2, \text{Setosa}]$
- Row vectors compress complex, multi-dimensional data into one compact object, easy to store, process, and model

Figure 4 · A Row Vector in the Iris Dataset

- Each row $\mathbf{x}^{(i)}$ represents one iris sample, living in a 4-dimensional feature space
- A row vector turns each sample into a single point in that 4D space

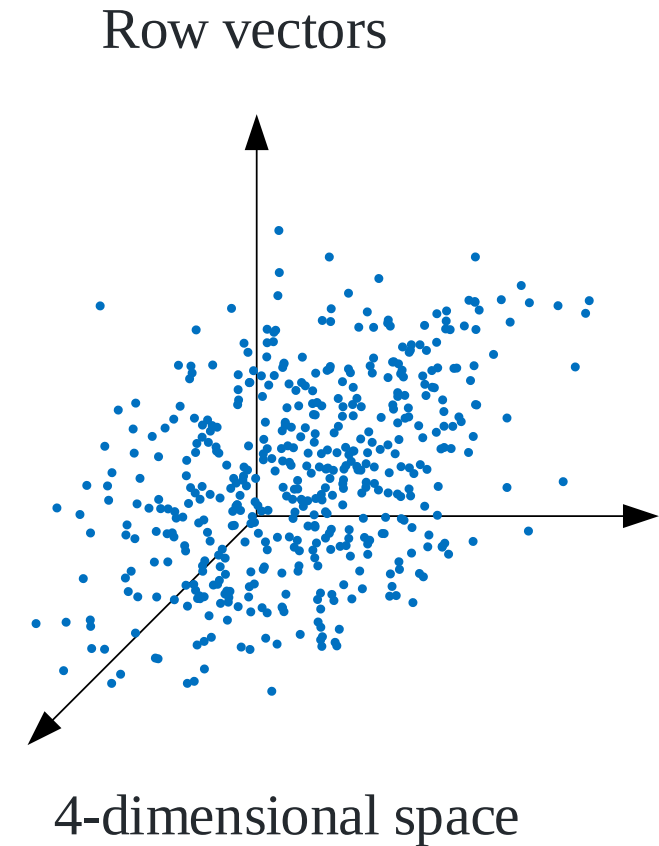
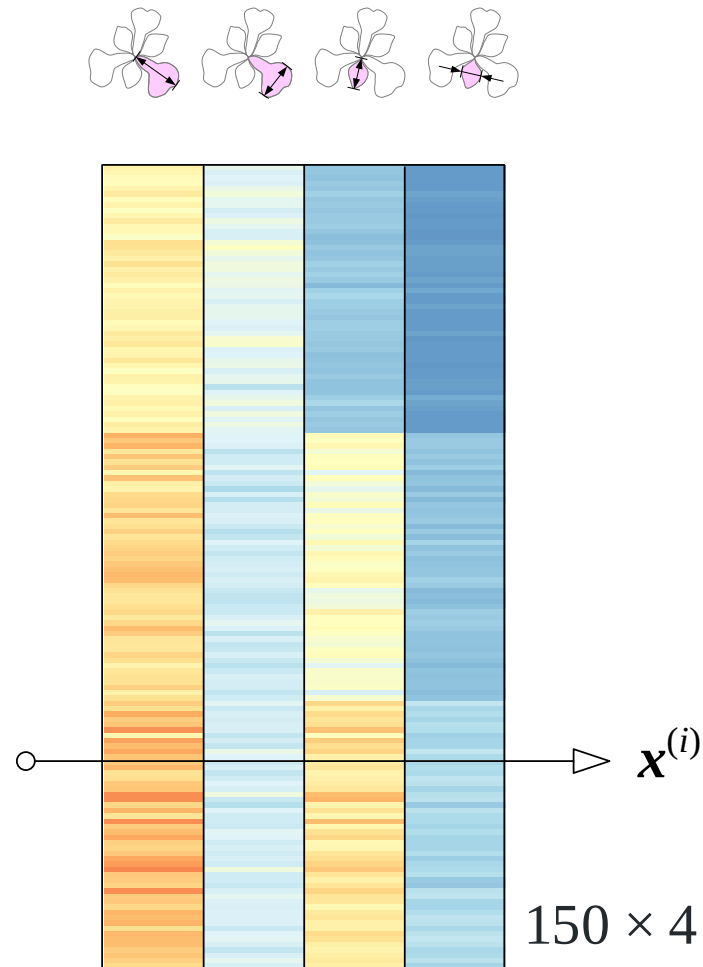
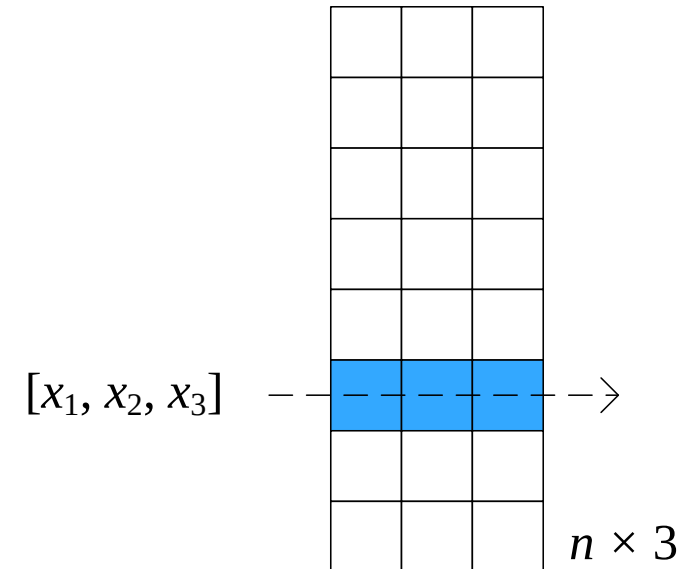
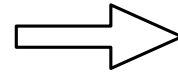
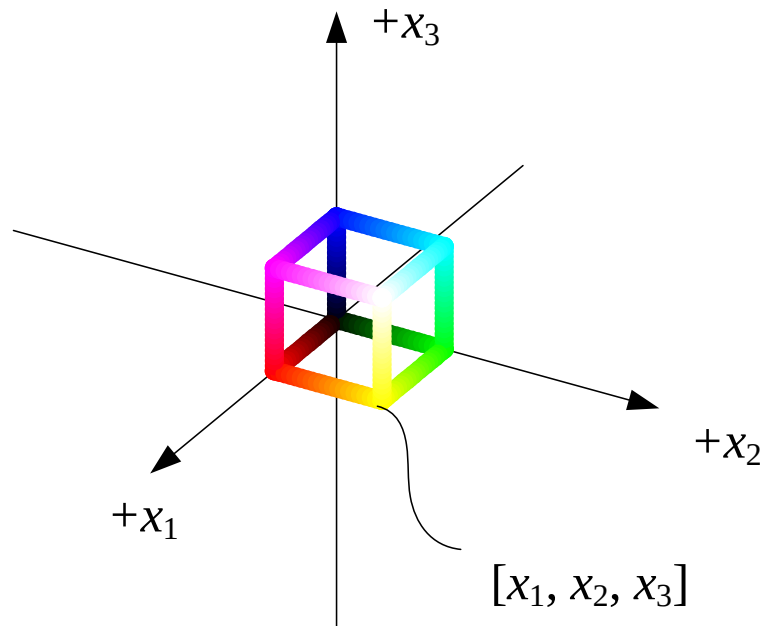


Figure 5 · The Wireframe of the RGB Color Cube

- Stacking n colors' row vectors together gives an $n \times 3$ rectangular matrix
- Each row is one color — the prototype of a rectangular matrix



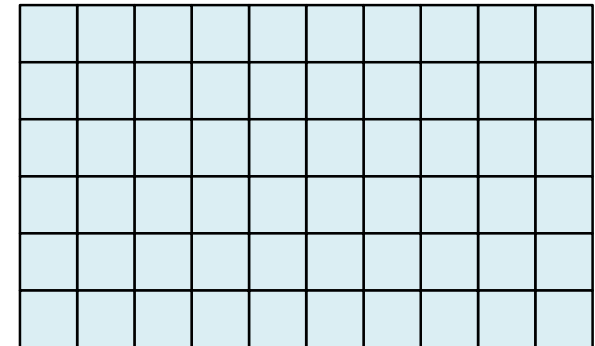
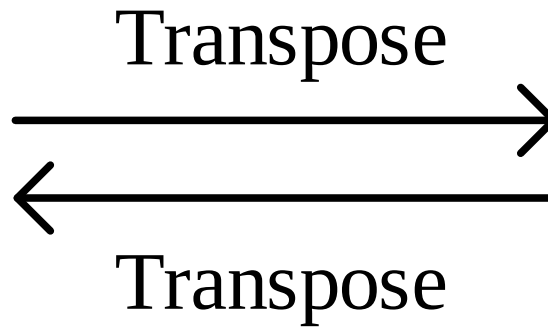
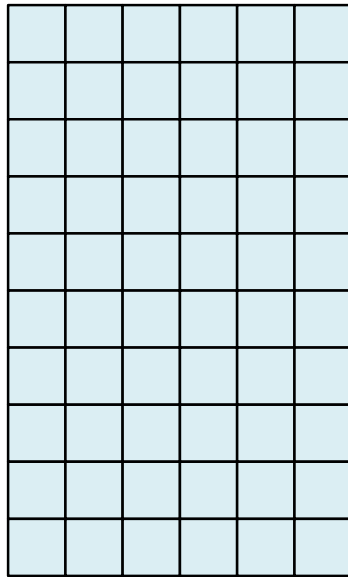
Rectangular Matrices: Tall and Wide

A tall matrix (far more rows than columns) and a wide matrix (far more columns than rows) transpose into each other

- A tall matrix often represents a dataset with many more samples than features, e.g. a 1000×10 data matrix
- A wide matrix often represents far more features than samples, e.g. a 100×10000 matrix in gene-expression data

Figure 6 · Tall and Wide Matrices Convert via Transpose

- A tall matrix transposes into a wide one, and vice versa
- This matches exactly the transpose relationship covered in the previous section



Square Matrix: Equal Rows and Columns

An $n \times n$ square matrix A has shape $n \times n$; many important operations require this shape

- A square matrix's main diagonal runs from top-left to bottom-right, splitting it into the diagonal plus two triangular regions
- Filling in zeros for missing entries yields symmetric, diagonal, and triangular matrices — special types of square matrices

Figure 7 · The Main Diagonal Splits a Square Matrix into Regions

- The main diagonal splits a square matrix into an upper triangle, the diagonal itself, and a lower triangle
- This split underlies special square matrices like symmetric and triangular matrices

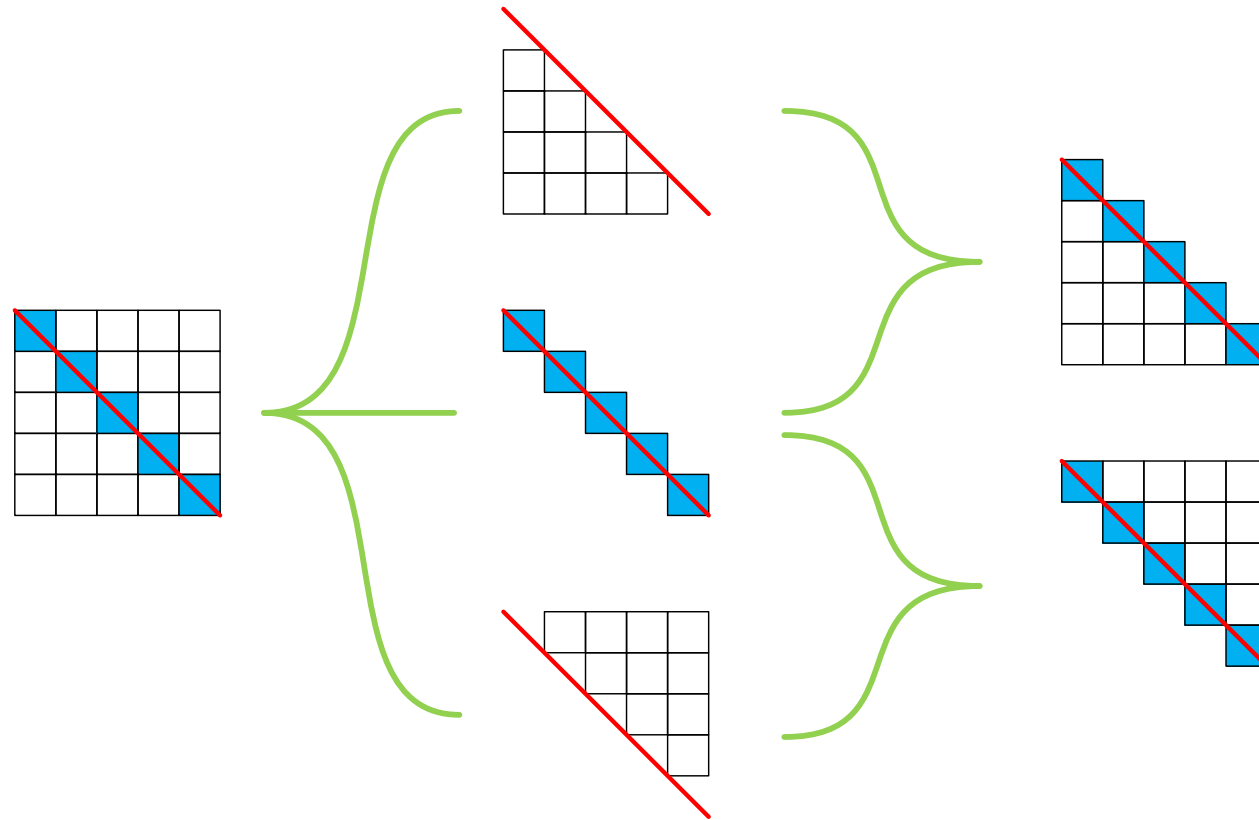


Table 1 · Matrix Operations That Require a Square Matrix

Determinant *Defined only for a square matrix; used to test invertibility (non-singularity) and more*

Matrix Inverse *Exists only when the matrix is square with nonzero determinant; a non-square matrix has a pseudo-inverse instead*

Eigendecomposition *Applies only to square matrices*

Diagonalization *Only a square matrix can be diagonalized — converted to a diagonal matrix via a similarity transform*

Cholesky Decomposition *Applies to a positive-definite matrix (which presumes a symmetric square matrix)*

Trace *The sum of a square matrix's main-diagonal entries — the overall "scaling effect" of its transform, or the sum of its eigenvalues*

Matrix Power *An integer power such as A^n generally requires a square matrix, or matrix multiplication can't proceed*

Spectral Decomposition *Applies only to symmetric square matrices*

LU Decomposition *Factors a square matrix into a lower-triangular and an upper-triangular matrix*

Quadratic Form, Rayleigh Quotient *A quadratic form presumes a square matrix; the Rayleigh quotient's numerator likewise needs one*

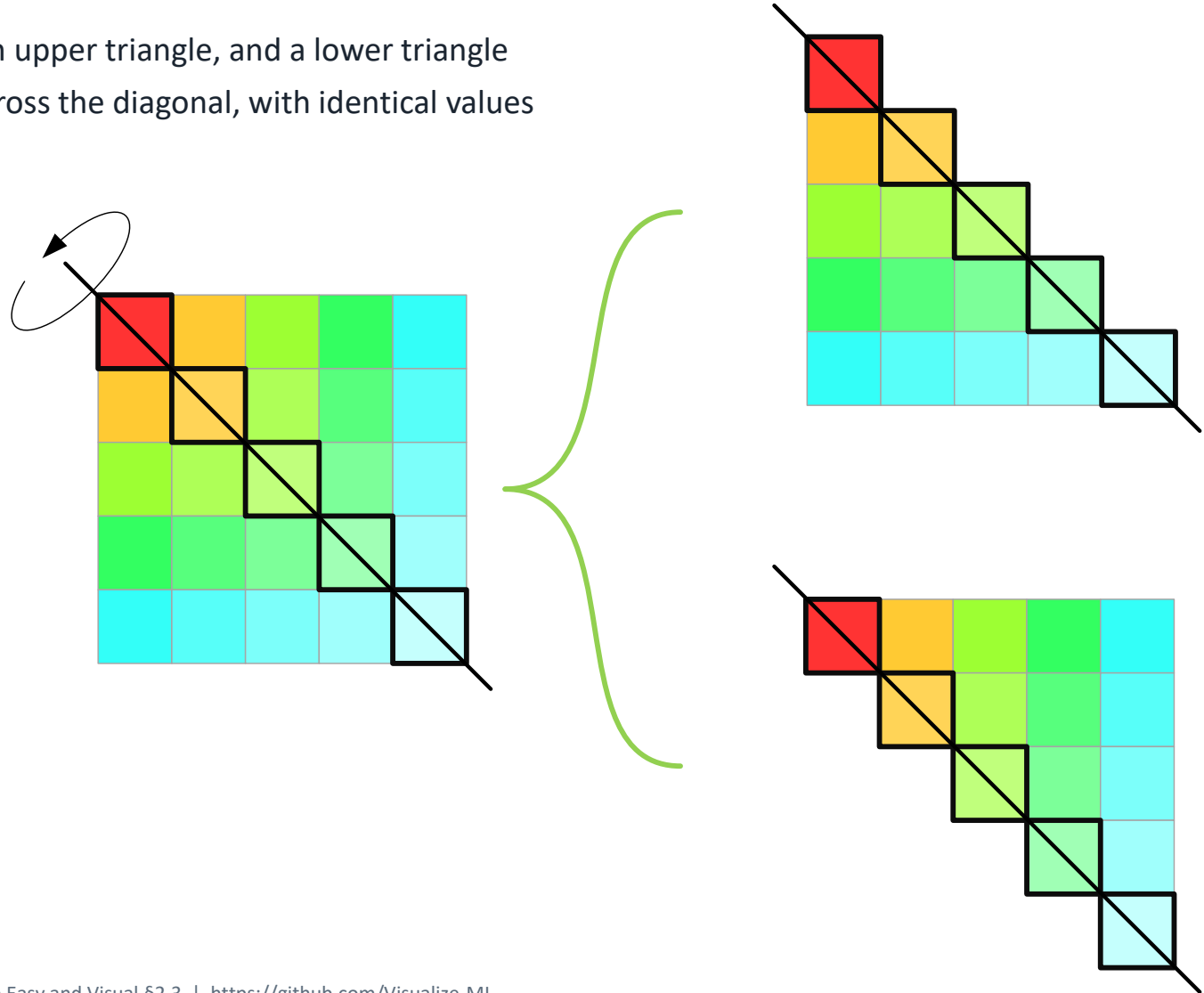
Symmetric Matrix: $A = A^T$

An $n \times n$ square matrix A satisfying $A = A^T$ is called symmetric

- Entries $a_{i,j}$ and $a_{j,i}$ are equal — entries mirror across the main diagonal
- A symmetric matrix is fully described by its main diagonal plus one triangle (upper or lower)

Figure 8 · Splitting a Symmetric Matrix

- A symmetric matrix splits into the main diagonal, an upper triangle, and a lower triangle
- The upper and lower triangles mirror each other across the diagonal, with identical values



Diagonal Matrix: Every Off-Diagonal Entry Is Zero

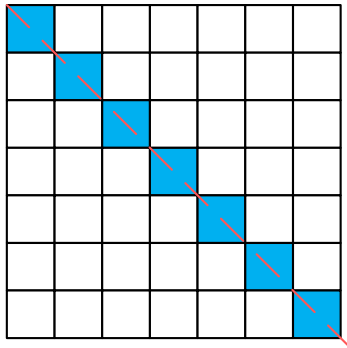
A diagonal matrix has all its nonzero entries on the main diagonal

- Main-diagonal entries may be zero or nonzero; a diagonal matrix can be either square or rectangular
- A tall diagonal matrix ($m > p$) or a wide one ($m < p$) still has every off-diagonal entry equal to zero

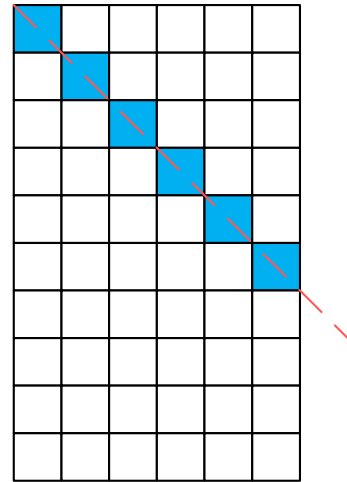
Figure 9 · Diagonal Matrices in Different Shapes

- (a) A square diagonal matrix: equal rows and columns
- (b)(c) Rectangular diagonal matrices: tall (rows>columns) or wide (rows<columns)

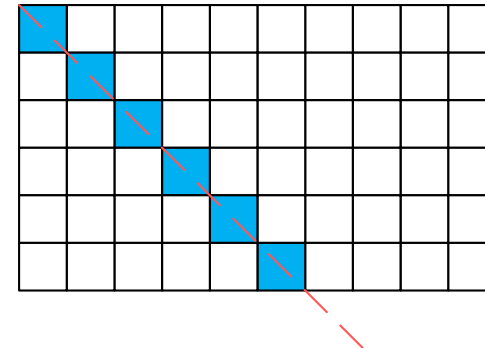
(a)



(b)



(c)



Identity Matrix: All 1s on the Main Diagonal

This book writes the identity matrix as I : $I_{n \times n}$ has 1s on the diagonal and 0s elsewhere

- The identity matrix is a special diagonal matrix — the "number 1" of matrix multiplication
- Every column of the identity matrix is a standard orthonormal basis vector; this book reserves E for that basis (not the identity matrix)

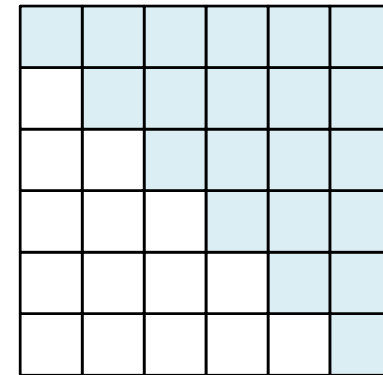
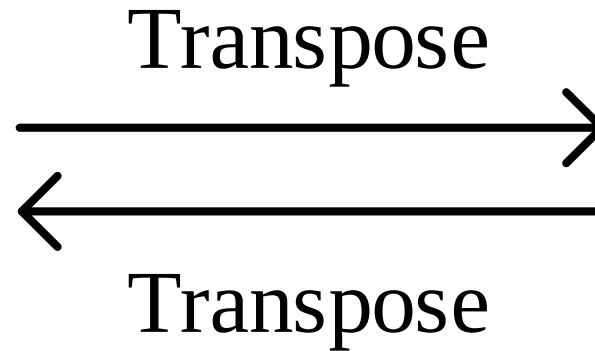
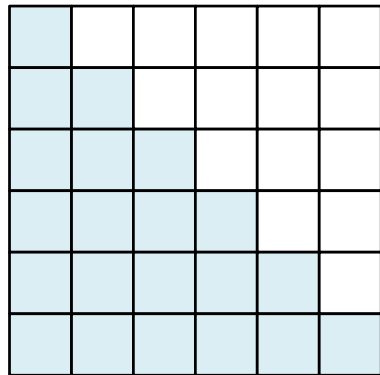
Triangular Matrix: Upper and Lower

Zero below the diagonal is an upper-triangular matrix U ; zero above the diagonal is a lower-triangular matrix L

- Upper- and lower-triangular matrices transpose into each other: $U^T = L$, $L^T = U$
- Triangular matrices simplify solving systems of linear equations, as in LU decomposition

Figure 10 · An Upper-Triangular Matrix Transposes into a Lower-Triangular One

- An upper-triangular matrix transposes into a lower-triangular one, and vice versa
- This follows the same transpose pattern as tall $\leftrightarrow$ wide matrices and symmetric matrices



Block Matrix: Cutting a Large Matrix into Smaller Ones

A block matrix slices a large matrix into several submatrices (blocks), like cutting a block of tofu

- Each smaller matrix is called a block or submatrix
- Unhappy with a matrix's shape? Block matrices let you cut it into the shape you want

Example: Cutting a 5×5 Matrix Top and Bottom

$$A = [A_1; A_2]$$

- Given a 5×5 matrix A (diagonal 3, 2, 1, zero elsewhere), one horizontal cut yields two submatrices
- A_1 is a 3×3 diagonal square matrix; A_2 is a 2×3 zero matrix

Example: Cutting a Matrix Both Ways

$$A = [A_{11}, A_{12}; A_{21}, A_{22}]$$

- One horizontal cut and one vertical cut split the matrix into four submatrices — the matrix's new "entries"
- This notation's drawback: it doesn't show each submatrix's shape directly, so those must be stated separately

Figure 11 · One Horizontal Cut

- A 5×12 matrix A , cut at rows 1:3 and 4:5, yields $A_{1:3, :}$ and $A_{4:5, :}$
- This slicing notation echoes NumPy indexing, but counts from 1

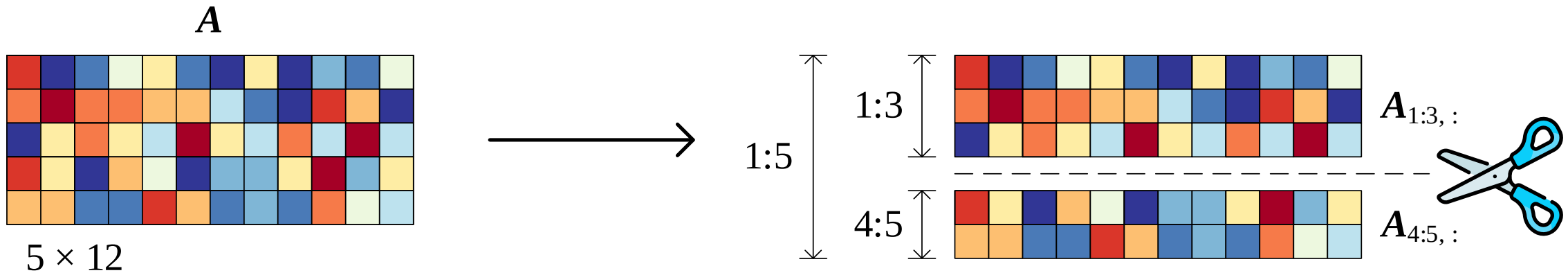


Figure 12 · One Vertical Cut

- Matrix A , cut at columns 1:5 and 6:12, yields $A[:, 1:5]$ and $A[:, 6:12]$
- Block matrices make computation on large matrices more efficient and easier to read

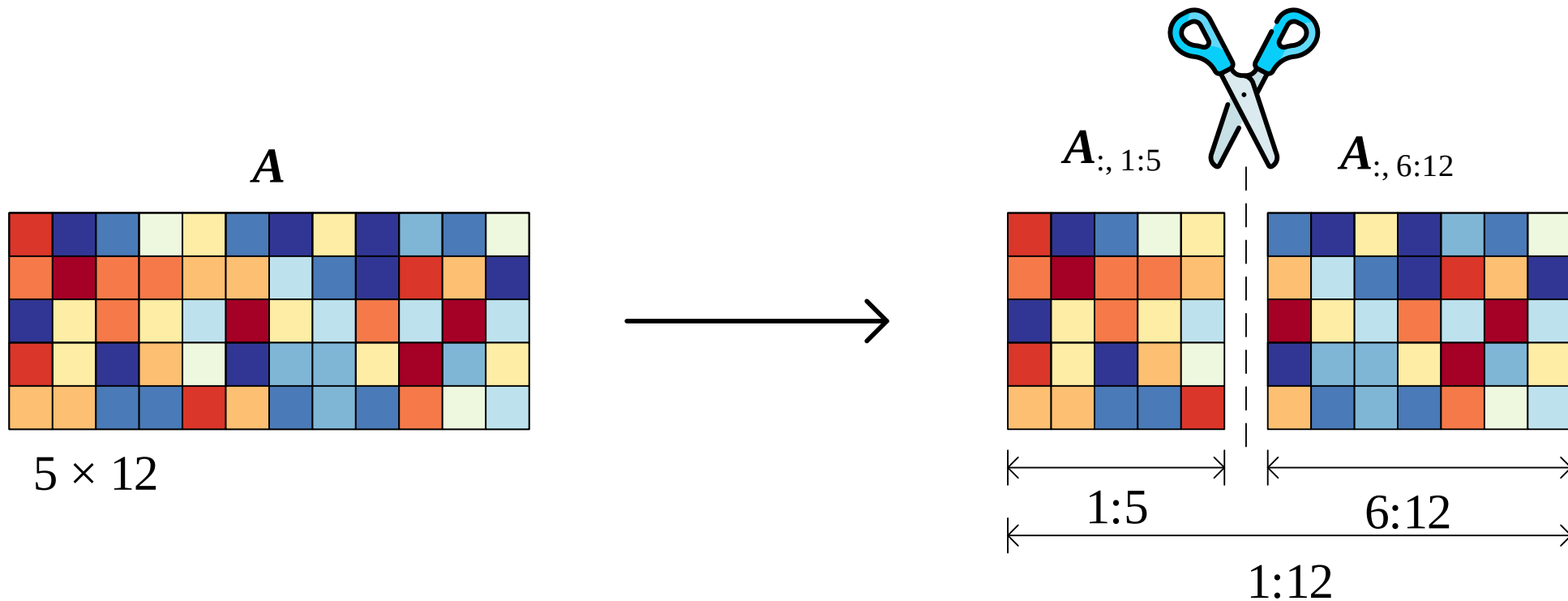
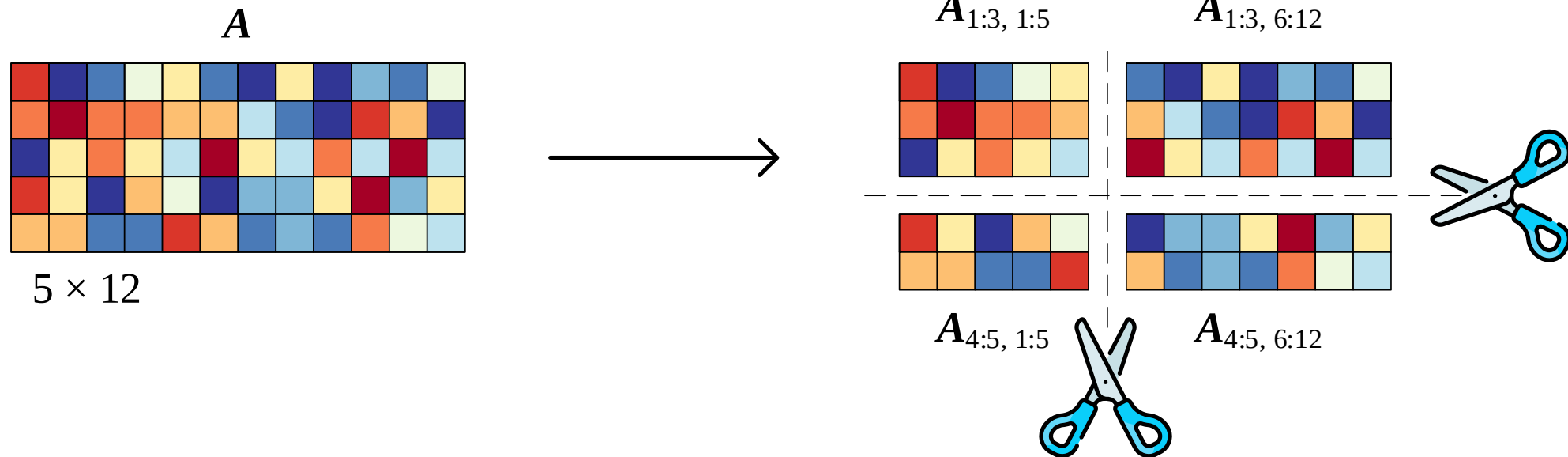


Figure 13 · One Horizontal Cut and One Vertical Cut

- Cutting A both ways yields four submatrices: $A_{1:3, 1:5}$, $A_{1:3, 6:12}$, $A_{4:5, 1:5}$, $A_{4:5, 6:12}$
- Later sections use block matrices to help explain matrix multiplication



Key Terms in This Section

Column Vector *A matrix with one column and many rows — this book's default vector form*

Rectangular Matrix *Unequal rows and columns — either tall or wide*

Symmetric Matrix *Satisfies $A=A^T$; entries mirror across the main diagonal*

Identity Matrix *A special diagonal matrix with all 1s on the diagonal, written I*

Block Matrix *A large matrix cut into several submatrices (blocks)*

Row Vector *A matrix with one row and many columns, often a single data sample*

Square Matrix *Equal rows and columns — a prerequisite for many matrix operations*

Diagonal Matrix *Every off-diagonal entry is zero; can be square or rectangular*

Triangular Matrix *Upper-triangular U and lower-triangular L transpose into each other*

Standard Orthonormal Basis *Written E in this book; each column of the identity matrix is one basis vector*

GO FURTHER

Companion Code and Further Reading

Further Reading · LA_02_03_02.ipynb

NumPy implementations of different matrix shapes — self-study

Recommended Reading · The Power of Matrices

A deeper look at where square-matrix operations (determinant, eigendecomposition) apply

Table 1 · Operations That Require a Square Matrix

Worth revisiting this quick-reference table after finishing the book

Open-source Repository

Code and image assets for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Use `numpy.eye()` to generate a 3×3 identity matrix, and check whether any two of its columns are orthogonal (dot product 0).

Q3 Write your own code (without `numpy.diag()`) to turn a given sequence of diagonal entries into a diagonal square matrix.

Q5 Learn to use `numpy.tril()` to extract a lower-triangular square matrix.

Q7 Use `numpy.random.randint()` to generate a 5×12 matrix, and cut it into submatrices as shown in Figures 11, 12, and 13.

Q9 Write functions `is_tall(A)` and `is_wide(A)` that determine whether an input matrix is tall or wide.

Q2 Write your own code (without `numpy.diag()`) to extract the main-diagonal entries of any matrix.

Q4 Learn to use `numpy.triu()` to extract an upper-triangular square matrix.

Q6 Write code that checks whether a matrix equals its own transpose to determine if it is symmetric. Use `numpy.array_equal()` to check whether two matrices match.

Q8 Generate a random 1000×10 matrix simulating a tall dataset, verify that its transpose has shape 10×1000 , and explain what data scenario this corresponds to.

Questions Q10 – Q18

Q10 Generate a random 6×6 matrix A with `numpy.random.randint()`, build $B=(A+A.T)/2$, and verify whether B is always symmetric.

Q12 Use `numpy.zeros()` and `numpy.fill_diagonal()` to build a 5×8 rectangular diagonal matrix, and visualize it as a heatmap.

Q14 Use NumPy to verify that every column of the identity matrix I is a standard orthonormal basis vector (hint: any two columns have dot product 0, and each column has norm 1).

Q16 Generate a random matrix with `numpy.random.randint()`, turn it into an upper-triangular matrix U with `numpy.triu()`, and verify whether $U.T$ is lower-triangular.

Q18 Use `numpy.linalg.det()` to compute the determinant of several square matrices, and observe what's special about a matrix whose determinant is 0 (hint: try constructing a singular matrix).

Q11 Write a function `is_diagonal(A)` that checks whether a square matrix is diagonal (hint: compare A against the diagonal matrix built from `numpy.diag(numpy.diag(A))`).

Q13 Write a function `make_identity(n)` that builds an $n \times n$ identity matrix by hand, without calling `numpy.eye()`.

Q15 Write functions `is_upper_triangular(A)` and `is_lower_triangular(A)` that check whether a square matrix is upper- or lower-triangular.

Q17 Write a function `trace(A)` that computes a square matrix's trace (the sum of its main-diagonal entries) by hand, without calling `numpy.trace()`.

Questions Q19 – Q27

Q19 Write a function `block_split_rows(A, k)` that cuts matrix `A` into two submatrices at row `k`, and returns their shapes.

Q21 Generate a random 8×8 matrix with `numpy.random.randint()`, cut it into four submatrices as in Figure 13, then reassemble it with `numpy.block()` and verify the result matches the original.

Q23 Load the Iris dataset with Pandas, convert it to a NumPy matrix, extract the column vector for sepal length, and compute its mean and variance.

Q25 Build a 5×5 block-diagonal matrix with two small square matrices on the main diagonal and zeros elsewhere, and visualize it as a heatmap.

Q27 Bringing it together, write a small program that, given any matrix, checks and reports whether it is a vector, square, symmetric, diagonal, or triangular, along with a description of its shape.

Q20 Write a function `block_split_cols(A, k)` that cuts matrix `A` into two submatrices at column `k`, and returns their shapes.

Q22 Write a function `matrix_shape_report(A)` that, given any matrix, reports whether it is square, symmetric, diagonal, or upper/lower triangular.

Q24 Use `seaborn.heatmap()` to draw a tall matrix and its transposed wide matrix side by side, comparing how their shapes differ.

Q26 Write code verifying that multiplying the identity matrix `I` by any matrix `A` of matching shape (with `numpy.matmul()` or `@`) returns `A` itself (hint: a preview of matrix multiplication, next section's topic).

10 Prompts to Explore Further (1/2)

1

Explain why many matrix operations (like the determinant and eigendecomposition) require a square matrix — what's the geometric or algebraic reason?

2

Use an everyday example to explain the difference between a tall matrix and a wide matrix — say, far more samples than features, or the reverse.

3

Prove that for any square matrix A , $B=(A+A^T)/2$ is always symmetric; what practical use does this construction have?

4

Explain why the identity matrix plays the role of "number 1" in matrix multiplication, and how that compares to the number 1 in scalar multiplication.

5

Explain why upper- and lower-triangular matrices simplify solving systems of linear equations, as in LU decomposition.

10 Prompts to Explore Further (2/2)

6

Design a small Python experiment verifying that a tall matrix always transposes into a wide one, and vice versa.

7

Explain the "cutting tofu" metaphor for block matrices, and why this notation has real value in large-scale matrix computation, such as deep learning.

8

Explain why a diagonal matrix can be either square or rectangular, and how that relates to the definition of a "diagonal."

9

Give a data-science scenario explaining why we'd want to cut a matrix into blocks — such as batch processing or parallel computation.

10

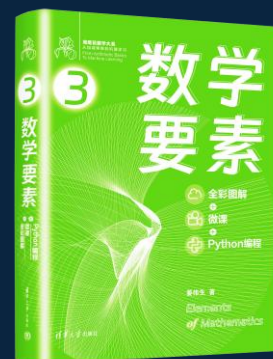
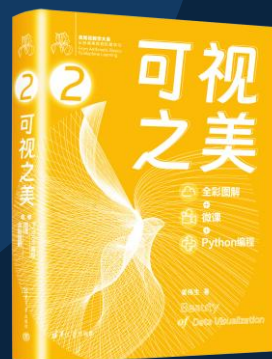
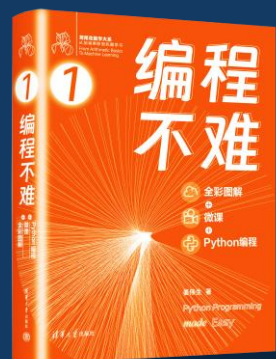
If a matrix is both diagonal and triangular, what other matrix categories might it also belong to? Construct an example in Python to verify.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

